10.2.4 Implement the step definitions

Writing a step definition is an exercise in code design. When you write the step definition, you come in contact with your application code for the first time. Although you may have some ideas about the high-level architecture and design, the step definition code is where the rubber meets the road and your design takes the form of real code for the first time.

> **NOTE** Because this chapter focuses on BDD and TDD unit-testing practices, we'll concentrate on acceptance criteria that manipulate the application code directly. Chapter 8 discusses automating acceptance criteria where a UI is involved.

Of course, not all steps are equal. Some step definitions contain relatively simple test code, whereas others involve more forethought and design effort.

In this case, the first step is very simple indeed:

```
String firstName;
String lastName;

@Given("^(\\S*) (\\S*) is not a Frequent Flyer member$")
    public void not_a_Frequent_Flyer_member(String firstName,
                                            String lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
}
```

❶ **Store the new member's name here.**

❷ **Note the name of the new member for use in the following step definitions.**

In the first step ❶, you need to ensure that Jill isn't already a Frequent Flyer member. There's little to do here except record the name of the prospective member so that you can reuse it in the subsequent steps.

The next step definition ❷ is a little more interesting, as you introduce the domain concept of the Frequent Flyer member. But before you start writing code, you'd typically make sure that you have a reasonable understanding of this domain concept.

10.2.5 Understand the domain model

Although incremental and emergent design works well in many situations, a team practicing BDD doesn't exclude more structured design practices where appropriate. In particular, BDD draws many concepts from Domain-Driven Design, including the idea of a common (or *ubiquitous*) language shared by everyone in the team, from business stakeholders to developers. This common language is founded on a high-level domain model, which can be built up incrementally as features are added, as you'll do in this example. This isn't the only option: many teams, particularly larger ones, often draw up a broader domain model supporting several features early on in the iteration. In a similar way, BDD teams will usually give some forethought to the high-level application design and architecture before the coding starts.

In our example, an important business entity has emerged: the Frequent Flyer member. From conversations with the business, you might learn that a Frequent Flyer